

Tecnológico de Monterrey

Campus Puebla

Course

Movilidad Inteligente (MR3004C.601)

Assignment

M4 Activity 2: Implementing a Pure Pursuit Controller in ROS 2

Professor

Arturo Daniel Sosa Cerón

Student

Gabriel Said Vera Carballo A00839097

GitHub repository

https://github.com/Gabrysaid/pure_pursuit_ws

Date

September 27, 2026

1. Implementation

We created the `ament_python` package `pure_pursuit_controller`, which contains two nodes and a ROS-free module called `pure_pursuit_core.py`. This module contains the main mathematical calculations, which allowed us to test them without running the simulator. We used 6 `pytest` cases to check yaw conversion, straight-line motion, turn direction, bicycle model mapping, and index monotonicity over two laps.

The **`path_recorder`** node subscribes to `/odom` (`nav_msgs/Odometry`) and stores the (x, y) position only when the Euclidean distance from the last stored point is greater than `min_distance = 0.5` m. The points are saved to a CSV file both from a `finally` block and an `on_shutdown` hook, so the file is still generated when the program is stopped with `Ctrl+C`. The same node can also be launched with a different output name to record the path followed during autonomous runs.

The **`pure_pursuit_node`** separates the odometry callback from the controller. The callback only stores the current state (x, y, θ , v) and its timestamp, while a 20 Hz timer runs the controller and publishes `/cmd_vel` using `geometry_msgs/Twist`. This keeps the controller update rate independent from the odometry rate. If the odometry data is older than 0.5 s, the node commands zero velocity. It also stops automatically after one lap and sends a final stop command when `Ctrl+C` is pressed. For this, the default `rclpy` signal handler is disabled. All controller gains are loaded as ROS parameters from `config/pure_pursuit.yaml` and can also be changed while the node is running, for example with `ros2 param set /pure_pursuit_node lookahead_gain 0.8`. The node also publishes the target point and the signed cross-track error to help with debugging.

The launch file `launch/pure_pursuit.launch.py` includes `prius_bringup/gz_sim.launch.py`. After a short delay, it starts the controller and the trajectory logger. The speed, look-ahead parameters, and output file name can be changed using command-line arguments. The reference path was recorded using `teleop_twist_keyboard` and `record_path.launch.py`, resulting in 374 waypoints over a total distance of 192 m.

2. Control formulation

The vehicle heading in the plane is obtained from the odometry quaternion using the ZYX convention:

$$\theta = \text{atan2}(2(q_w q_z + q_x q_y), 1 - 2(q_y^2 + q_z^2))$$

The look-ahead distance changes with the vehicle speed according to $L_d = L_{d,\min} + k \cdot v$, with a maximum value of 15 m. To avoid jumping between different parts of the path, we search for the closest waypoint only in the forward direction from the previous waypoint and within a 60-point window. An unwrapped index is used so the controller can pass through the end of the closed path without going back to index 0. From the closest point, we move forward until finding the first waypoint whose distance from the vehicle is at least L_d . This waypoint is used as the target point (x_t, y_t). The angle to the target, $\alpha = \text{atan2}(y_t - y, x_t - x) - \theta$, is wrapped to the interval $(-\pi, \pi]$. Using this angle, the curvature of the arc between the vehicle and the target point is $\kappa = 2 \sin \alpha / L_d$. For the bicycle model, the relationship between curvature and steering angle is $\tan \delta = L \kappa$. Therefore, the steering angle used by the controller is:

$$\delta = \tan^{-1}(2L \sin \alpha / L_d), \quad |\delta| \leq 0.5 \text{ rad}$$

Since $d\theta/dt = v\kappa$, the yaw rate sent as `angular.z` is:

$$d\theta/dt = v \tan \delta / L$$

The wheelbase $L = 2.7$ m and the 0.5 rad steering limit were taken from the `<wheel_base>` and `<steering_limit>` tags in the Prius AckermannSteering plugin. The plugin uses the same value of L when converting the command, so the steering angle calculated by the controller is the same one applied to the front wheels. The commanded speed is used in the yaw-rate equation (3), while the

measured speed is used to calculate L_d . The default parameters are $v = 4$ m/s, $L_{d,\min} = 3$ m, and $k = 0.4$ s.

3. Tracking performance

The cross-track error (CTE) is calculated as the distance between each logged point and the closest segment of the reference path. This calculation is done in `scripts/analyze_tracking.py`. To separate curves from straight sections, we calculated the curvature of the reference path using a 10 m stencil. Points with a curvature greater than 0.02 m^{-1} , equivalent to $R < 50$ m, were classified as curves. The remaining points were classified as straights. All results were obtained from Gazebo Sim 10, Prius model at 4 m/s, comparing the autonomous trajectory with the recorded reference path of 374 waypoints over a total distance of 192 m. The teleoperated path was trimmed at the point where it returned to the starting straight, leaving a 0.19 m gap. This creates a sharp connection when the loop is closed.

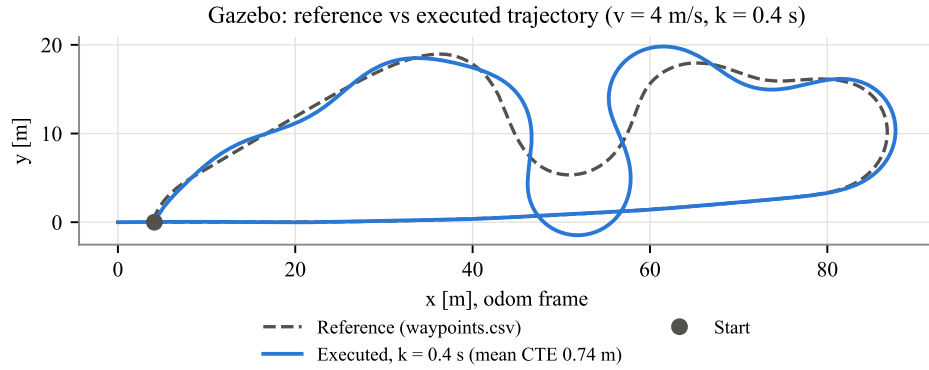


Fig. 1. Reference path (waypoints.csv) against the executed trajectory (actual_trajectory_main.csv) in the odometry frame, with equal axis scale.

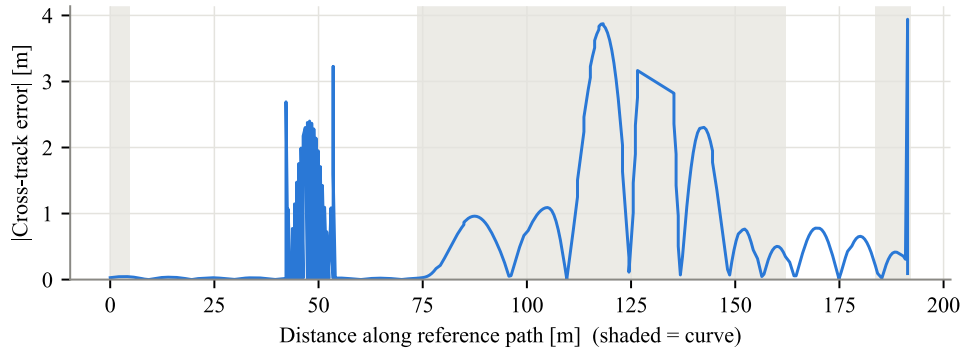


Fig. 2. Absolute CTE along the reference path ($k = 0.4$ s). Shaded regions represent curves. The spikes near 45 m correspond to points from the S-bend excursion that are closest to the starting straight.

For the default parameters, the mean CTE over the complete lap was 0.74 m, with an RMS error of 1.22 m and a maximum error of 3.94 m. On straight sections, the mean CTE was 0.36 m, while on curves it increased to 1.11 m, which is 3.1 times higher. There are two main reasons for this difference. The first one is corner cutting. Since the target point is located L_d ahead of the vehicle, it can already be inside a curve before the vehicle reaches it. This makes the controller start turning earlier and follow a chord inside the curve instead of following the reference path exactly. The second factor is steering lag. When we commanded $\omega = 0.3$ rad/s in Gazebo, the odometry reported only 0.275 rad/s after the transient. This happens because the steering joints need some time to reach the commanded angle. The effect becomes more important in the tight S-bend, where the radius is approximately 6 m. This is close to the minimum radius of $L/\tan(0.5) = 4.9$ m. Because of the steering limit, the vehicle saturates the

steering, overshoots the path, and makes one loop before returning to the reference. This produces the maximum tracking error.

4. Look-ahead tuning

We tested different values of the look-ahead gain k , keeping $L_{d,\min} = 3$ m and the speed at 4 m/s. Each value was tested in one Gazebo run, and the simulation was restarted before every run.

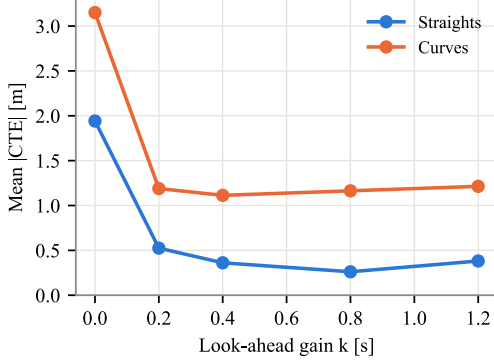


Fig. 3. Mean CTE versus look-ahead gain k ($L_{d,\min} = 3$ m, $v = 4$ m/s), Gazebo.

Run	Mean	RMS	P95	Max	Straight	Curve
$k = 0.0$	2.57	3.79	8.44	10.49	1.94	3.15
$k = 0.2$	0.86	1.28	2.87	3.94	0.52	1.19
$k = 0.4$	0.74	1.22	3.05	3.94	0.36	1.11
$k = 0.8$	0.72	1.18	2.90	3.97	0.26	1.16
$k = 1.2$	0.82	1.30	2.95	4.21	0.38	1.21

Table 1. CTE in metres. One Gazebo run was performed for each value of k , with the simulation restarted before each run.

With a short look-ahead, such as $k = 0$, the look-ahead distance stays at 3 m. This makes the geometric gain $2L/L_d^2$ relatively high. Together with the steering lag, this causes the vehicle to overshoot the corners and loop around the path. The mean CTE in this case was 2.57 m and the maximum error was 10.49 m. The vehicle also stopped before completing the lap because the forward index continued advancing while the vehicle was looping.

Increasing k reduces this behavior. At $k = 0.2$ s, the mean CTE decreased to 0.86 m. Between $k = 0.4$ and $k = 0.8$ s, the mean error changed only slightly, from 0.74 to 0.72 m. During this range, the longer look-ahead improved the performance on straight sections, reducing the error from 0.36 to 0.26 m, but the curve error increased from 1.11 to 1.16 m. At $k = 1.2$ s, the target point is farther ahead and the controller starts cutting the inside of the corners more. As a result, the curve error increases to 1.21 m and the mean CTE increases to 0.82 m.

Because L_d is proportional to speed, the preview time remains approximately constant. This means the same value of k can be used at different speeds. The lowest mean CTE in the tests was obtained with $k = 0.8$ s, but the difference between $k = 0.8$ and $k = 0.4$ was only 0.02 m, which is smaller than what can be reliably distinguished from a single run. For this reason, we selected $k = 0.4$ s with $L_{d,\min} = 3$ m. This configuration gave a lower curve error and a lower maximum error while keeping a good balance between oscillation and corner cutting.

Conclusion. The package builds successfully with `colcon` without warnings and runs the controller using a fixed-rate timer. At 4 m/s, the controller was able to reproduce the recorded path with a mean tracking error of 0.74 m. The main limitation was the tight S-bend, where the steering limit and the delay in the steering response had the greatest effect on the trajectory. A speed profile that changes according to the curvature or the addition of a Stanley term for lateral error could help reduce the tracking error in this section.

References

- [1] R. C. Coulter, "Implementation of the Pure Pursuit Path Tracking Algorithm," Carnegie Mellon University, Robotics Institute, Tech. Rep. CMU-RI-TR-92-01, 1992.
- [2] J. M. Snider, "Automatic Steering Methods for Autonomous Automobile Path Tracking," Carnegie Mellon University, Robotics Institute, Tech. Rep. CMU-RI-TR-09-08, 2009.
- [3] R. Rajamani, *Vehicle Dynamics and Control*, 2nd ed. New York: Springer, 2012.
- [4] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, 2022.
- [5] Open Robotics, "ROS 2 Lyrical Luth documentation," docs.ros.org/en/lyrical, 2026.
- [6] Open Robotics, "Gazebo Sim AckermannSteering system," gazebo.org/api/sim/10, 2026.
- [7] A. D. Sosa Cerón, "movilidad_inteligente" course repository (prius_bringup), github.com/dsosa114/movilidad_inteligente, 2026.
- [8] Anthropic, Claude [AI assistant], claude.ai, 2026. Used mainly for translation of the report and for programming support (code drafting, debugging and simulation scripts). The author recorded the reference path by teleoperation and reviewed the results and the final content.